FYS-8604: Multi-Modal Learning Home Exam Answer

Candidate ID: 7429216*

Instructors: Michael Kampffmeyer, Kristoffer Wickstrøm, and Rwiddhi Chakraborty

Abstract

The report presents the implementation and analysis of two problems from the FYS-8604 home exam. The first problem involves an exploration of a multi-modal digits classification challenge using different data types, such as visual and audio, as well as a fusion approach combining them. The second problem focuses on the theoretical aspect, exploring self-supervised loss functions such as the normalized temperature-scaled cross-entropy (NT-Xent). Disclaimer: Parts of the text and structure in this report were refined with the assistance of GPT.

1 Digits Classification

All implementations are executed on a MacBook Pro with an M3 Max chip, featuring 14 CPU cores and 30 GPU cores (Apple MPS). The code is developed using Python 3.10 and PyTorch 2.7.1.

For this practice, a simple Convolutional Neural Network (referred to as simpleCNN in the code), introduced in the course exercises, is utilized. It consists of two convolutional layers with 3×3 kernels, padding enabled, and 2×2 max pooling, followed by a fully connected output layer. ReLU is used as the activation function for each convolutional layer, and Softmax is applied at the output layer. The model is designed for grayscale (single-channel) image inputs. To ensure flexibility and uniformity in the latent space, a projecting layer is added, which maps the input features to a specified dimension using a lazy-initialized linear transformation followed by a ReLU activation. The chosen projection dimension is 4096, and the model is trained for 5 epochs using a learning rate of 0.00001, the Adam optimizer, and cross-entropy loss.

Besides the simpleCNN model, a pre-trained ConvNeXt model is also used to compare the performance of a basic network with that of a large pre-trained model. ConvNeXt is a modern CNN architecture that reimagines convolutional networks using design strategies inspired by Transformer models [1]. It incorporates Layer Normalization, GELU activation, large kernel sizes, and a patch-based stem—similar to Vision Transformers. This model is available in the `timm` Python package, which is integrated with Hugging Face. Like many modern pre-trained image classification models (including ResNet, ViT, and EfficientNet), ConvNeXt is trained on the ImageNet dataset, where all input images are resized to a standard resolution of 128×128 pixels with 3 RGB channels. Consequently, an additional preprocessing step is required to resize all of the input images in this assessment to a resolution of $3 \times 128 \times 128$.

Regarding data preparation, after applying modality-specific preprocessing, the data is either kept in its original size or interpolated to upscale it to 128×128. More specifically, bilinear interpolation is used to resize images to 128×128 via `torch.nn.functional.interpolate`. This method calculates each new pixel as a weighted average of the four nearest pixels, preserving smooth gradients and fine details in the image. The original and interpolated datasets are used to evaluate the effect of spatial resolution on model performance.

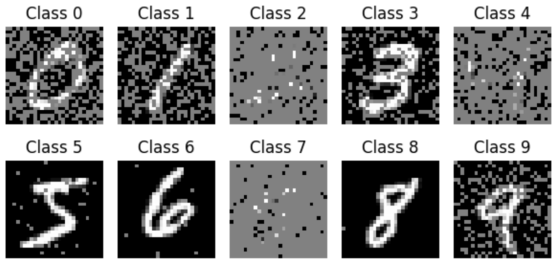
1.1 Image-based Classifier

By plotting image samples for each digit, it becomes evident that the images are affected by various types of noise, including salt-and-pepper noise (also known as impulse noise). This type of noise is introduced by randomly scattering white and black pixels throughout the image. Additionally, some images exhibit Gaussian-like background noise, where pixel intensities are randomly jittered. For example, in Figure 1(a), digits 0, 1, 3, 4, 6, 8, and 9 display salt-and-pepper noise, while digits 5 and 8 appear to have a strong gray overlay, possibly due to Gaussian noise. Figure 1 also shows that the training and test sets exhibit contrasting noise patterns: digits 2, 4, and 7 are difficult to recognize and digits 5, 6, and 8 have less noise in the training set, whereas the test set shows the opposite trend.

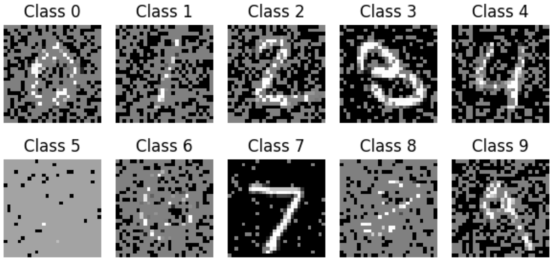
Table 1 shows the results of training unimodal-based classifiers—the simple CNN and the pre-trained ConvNeXt model—using both the original data size and the resized version obtained through interpolation. Specifically, the simpleCNN trained on the original data achieved a training accuracy of 79.04% and a test accuracy of 74.93%, with a training time of 11.83 seconds. When trained on the interpolated (1-channel) data, the same model improved significantly, reaching 92.37% training ac-

*Corresponding Author.

Proceedings of the 7th Northern Lights Deep Learning Conference (NLDL), PMLR V, 2026.



(a) Digit samples in Training set.



(b) Digit samples in Test set.

Figure 1. Image digits samples in Training and Test set.

Table 1. Result of Visual-based Classifiers in terms of accuracy (%).

Modality	Data Type	Model	Train Acc	Test Acc	Training Time
Visual	Original Data	Simple CNN	79.04%	74.93%	11.83s
Visual	Interpolated 1-Channel	Simple CNN	92.37%	80.32%	190.91s
Visual	Interpolated 3-Channel	ConvNeXt Tiny	97.23%	90.68%	400.54s

curacy and 80.32% test accuracy, though the training time increased to 190.91 seconds. The pre-trained ConvNeXt model, trained on interpolated 3-channel data, outperformed both simpleCNN configurations with a training accuracy of 97.23% and a test accuracy of 90.68%, at the cost of a significant longer training time of 400.54 seconds.

Figure 2 illustrates the training trends over 5 epochs, where test accuracy increases and training loss decreases. The training loss reflects the model's error on the training set after each epoch; the lower the training loss, the better the model's performance. Conversely, higher test accuracy—indicating the model's ability to correctly classify digits in the test set—also signifies better performance.

From these results, two key conclusions can be drawn: (1) the pre-trained ConvNeXt model achieves the highest overall accuracy, and (2) resizing the data using the interpolation method improves performance, even for the simple CNN. However, this improved performance comes at the cost of significantly increased processing time.

1.2 Audio-based Classifier

Instead of working with raw waveforms, audio data is transformed into its Mel spectrogram, which more closely reflects how humans perceive sound in terms

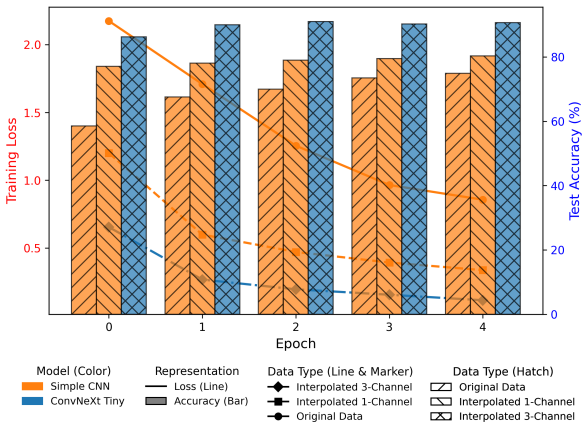
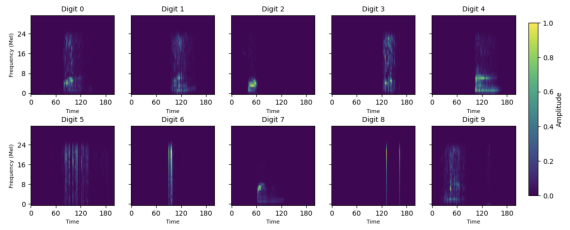


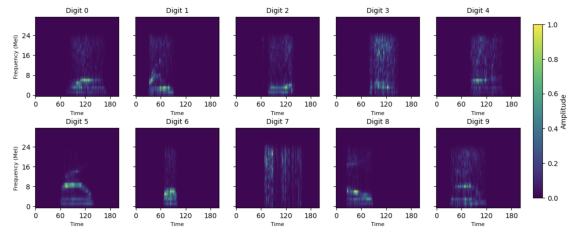
Figure 2. Training Loss (Line) and Test Accuracy (Bar) per Epoch

of frequency and amplitude. Each raw audio waveform is converted into a Mel spectrogram using `torchaudio.transforms.MelSpectrogram`, which represents the audio's frequency content on a perceptually relevant Mel scale. The transformation is configured with a resample rate of 16 kHz, an FFT (Fast Fourier Transform) window size of 256, a hop length of 80, and 32 Mel-scaled frequency bands. Since audio signals change over time, FFT is not applied to the entire signal at once, but to small overlapping chunks (windows). Initially, applying FFT results in a spectrogram, a grid showing time versus frequency with a linear scale. However, humans do not perceive all frequencies equally, so this linear scale is mapped to the Mel-scale, a non-linear frequency scale that better mimics human hearing. The FFT output is passed through a filter bank to produce 32 Mel-scaled frequency bands (bins), where each bin aggregates a range of FFT frequencies into a single value.

The result of the Mel-spectrogram transformation is a 2D tensor, where one axis represents time and the other corresponds to Mel-scaled frequency bands. Each value in the matrix reflects the energy at a specific time and frequency band. This format captures both the spectral and temporal characteristics of the audio, making it well-suited for machine learning tasks. Example output samples for each digit in the training and test sets are shown in Figure 3. From the spectrogram visualization, it is observed that the audio has been altered using techniques such as frequency masking, time masking, and amplitude noise injection to simulate real-world distortions. In several training samples, particularly classes like 6 and 8, there are noticeable gaps or missing bands along the Mel-frequency axis, which is a hallmark of frequency band removal. Similarly, classes such as 2, 7, and 9 show disruptions along the time axis, indicative of time-domain masking or cropping. Additionally, some training examples display irregular,



(a) Audio Mel-spectrogram samples in Training set.



(b) Audio Mel-spectrogram samples in Test set.

Figure 3. Audio Mel-spectrogram sample in Training and Test set.

Table 2. Result of Audio-based Classifiers in terms of accuracy (%).

Modality	Data Type	Model	Train Acc	Test Acc	Training Time
Audio	Original Data	Simple CNN	57.77%	49.43%	64.44s
Audio	Interpolated 1-Channel	Simple CNN	59.38%	51.34%	186.68s
Audio	Interpolated 3-Channel	ConvNeXt Tiny	96.63%	85.19%	389.99s

bright streaks or uneven energy distribution, suggesting the addition of amplitude noise to simulate environmental or recording noise. In contrast, the test set appears cleaner, with more consistent and complete spectrogram patterns. Hypothetically, this contrast implies that the training data was intentionally corrupted to encourage the model to learn more robust and noise-invariant representations.

Similarly to Section 1.1, the Mel-spectrogram data is fed into two models, the simpleCNN and the pre-trained ConvNeXt, and the input is rescaled using the binary interpolation method. The results are shown in Figure 4 and Table 2. They indicate that the audio-based classifier follows the same trend as the visual-based classifier: the pre-trained model delivers the best performance across all test cases, and interpolation helps improve accuracy. Specifically, the simple CNN achieves 49.43% test accuracy on the original data, which increases to 51.34% after interpolation. The ConvNeXt model significantly outperforms both, reaching 85.19% test accuracy. However, this improvement comes at a cost in training time, which takes up to 189.68 seconds and 389.99 seconds to train the simpleCNN and ConvNeXt, accordingly.

1.3 Uni-modality Comparison

Table 3 combines the results from uni-modality classifiers from the visual-based and the audio-based.

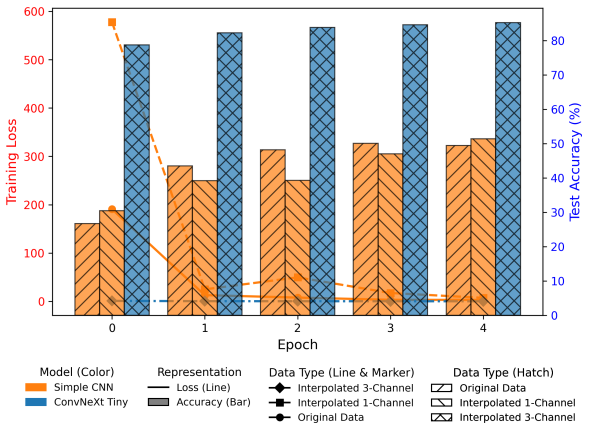


Figure 4. Training Loss (Line) and Test Accuracy (Bar) per Epoch

It shows that pre-trained ConvNeXt models consistently outperform the simple CNNs, regardless of the data modality or preprocessing. For both visual and audio inputs, interpolation significantly improves accuracy, particularly when paired with ConvNeXt. For instance, the visual ConvNeXt model reaches the highest test accuracy of 90.68%, followed closely by the audio ConvNeXt at 85.19%. Some extracted patterns and hypotheses regarding model performance across visual and audio modalities can be listed as:

- Visual data generally performs better than audio when using the same architecture and preprocessing setup. For example, even the simple CNN with original visual data achieves 76.45%, compared to only 49.77% for audio. This suggests that the visual modality may contain more distinguishable or cleaner features than from audio spectrograms.

- Interpolation helps, but at a significant computational cost. While accuracy improves with the test set, training time increases drastically. The impact is even more pronounced for pre-trained ConvNeXt model, which takes approximately 400 seconds after interpolation. Due to increased input resolution or channel depth, it can affect convolutional memory and processing time.

- Audio ConvNeXt generalizes extremely well, achieving 96.63% training accuracy and 85.19% test accuracy, despite the audio modality suspected being noisier and more variable. This suggests that large, pre-trained models are more capable of learning robust representations from corrupted or augmented data.

In summary, the results highlight the value of data interpolation and pre-trained models, while also illustrating the challenge of audio classification

Table 3. Result of Uni-modality Classifiers in terms of accuracy (%).

Modality	Data Type	Model	Train Acc	Test Acc	Training Time
Visual	Original Data	Simple CNN	79.04%	74.93%	11.83s
Visual	Interpolated 1-Channel	Simple CNN	92.37%	80.32%	190.91s
Visual	Interpolated 3-Channel	ConvNeXt Tiny	97.23%	90.68%	400.54s
Audio	Original Data	Simple CNN	57.77%	49.43%	64.44s
Audio	Interpolated 1-Channel	Simple CNN	59.38%	51.34%	186.68s
Audio	Interpolated 3-Channel	ConvNeXt Tiny	96.63%	85.19%	389.99s

relative to visual. The computational cost of interpolation is high, but the performance gains may justify it in many cases. These patterns re-confirm that modality-specific preprocessing and model selection play a critical role in classification performance.

1.4 Multi-Modal Learning

The answer for this part is extracted from the course material and the paper "Deep Multimodal Data Fusion" by Fei Zhao et. al.

In the early days, AI primarily focused on finding the most efficient representations for single or uni-modal data to solve specific problems. With the evolution of hardware technology, approaches that consider multiple uni-modal data sources have become increasingly promising. This trend is referred to as Multi-modal Learning (MML). Its purpose is to mimic the way humans interact with the world—gathering information from different sensory systems, such as sight and sound, and processing them through the brain. Generally, a typical MML system consists of three core components: an encoder, which transforms raw data into a new representation; a fusion method, which merges multimodal information into a unified representation; and a loss function, which evaluates performance. There is no fixed rule of how to place these components but rather the order of these components are flexible and depended on the application or model architecture.

One of the key components in Multi-modal Learning (MML) is the fusion block, traditionally used to generate complementary information from multiple modalities before making a final decision. There are various ways to classify fusion strategies, with one of the most common being the early-to-late fusion taxonomy. This framework includes early fusion, where modalities are combined before being fed into the model; intermediate fusion, which merges features after each modality has undergone individual feature extraction; late fusion, where the outputs or decisions of separate uni-modal models are combined; and hybrid fusion, which mixes fusion strategies at different stages of the pipeline.

Later, [2] proposed a more refined and fine-grained taxonomy that goes beyond this conventional setup, reflecting the evolution of state-of-the-art methods at the time. Their approach redefines fusion not just as an explicit architectural block, but also as an implicit mechanism embedded throughout the model. Fusion methods are categorized into Encoder-Decoder, Attention Mechanism, Graph Neural Network, Generative Neural Network, and other Constraint-based methods. This newer perspective reflects the increasing sophistication of modern MML systems, where fusion plays a more dynamic and integrated role in representation learning.

Table 4. Result of Feature Fusion-based Classifiers in terms of accuracy (%).

Modality	Data Type	Model	Train Acc	Test Acc	Training Time
Fused-Concat	Interpolated 3-Channel	ConvNeXt Tiny	100.00%	99.18%	844.97s
Fused-Add	Interpolated 3-Channel	ConvNeXt Tiny	99.77%	98.19%	823.31s
Fused-Multiply	Interpolated 3-Channel	ConvNeXt Tiny	100.00%	99.42%	830.48s
Fused-Average	Interpolated 3-Channel	ConvNeXt Tiny	100.00%	99.57%	830.93s
Fused-WeightedSum	Interpolated 3-Channel	ConvNeXt Tiny	100.00%	99.47%	825.23s
Fused-Gated	Interpolated 3-Channel	ConvNeXt Tiny	100.00%	99.32%	837.27s
Fused-Concat	Interpolated 1-Channel	Simple CNN	88.91%	79.50%	385.59s
Fused-Add	Interpolated 1-Channel	Simple CNN	90.74%	80.66%	374.47s
Fused-Multiply	Interpolated 1-Channel	Simple CNN	89.28%	83.89%	373.06s
Fused-Average	Interpolated 1-Channel	Simple CNN	91.12%	81.23%	372.32s
Fused-WeightedSum	Interpolated 1-Channel	Simple CNN	90.34%	81.33%	371.63s
Fused-Gated	Interpolated 1-Channel	Simple CNN	92.98%	83.21%	390.00s
Fused-Concat	Original Data	Simple CNN	85.36%	75.95%	74.99s
Fused-Add	Original Data	Simple CNN	82.55%	76.22%	72.87s
Fused-Multiply	Original Data	Simple CNN	92.44%	86.89%	72.66s
Fused-Average	Original Data	Simple CNN	85.32%	77.78%	72.53s
Fused-WeightedSum	Original Data	Simple CNN	88.13%	80.03%	72.70s
Fused-Gated	Original Data	Simple CNN	83.69%	77.78%	89.01s

This course introduces the another basic fusion taxonomy: feature fusion and score fusion. Feature fusion is the simplest form of early or intermediate fusion, where modality features are combined using straightforward operations such as concatenation or addition. Score fusion, on the other hand, is a common form of late fusion that combines the output scores or decisions from each modality. To enable a simple, learnable implementation of fusion, a linear layer is used, allowing the model to automatically learn how to weight and combine the multi-modal features effectively. This practice tries to examine both approaches with different combination operations. Section 1.5 and Section 1.6 provide more details.

1.5 Feature Fusion-based Classifier

Firstly, this section implements the feature fusion logic, supporting several basic and learnable strategies to combine audio and visual modality outputs. Simple fusing approaches include concatenation, summation, element-wise multiplication, and averaging, each combining features in a straightforward, non-parametric way. In addition, the weighted sum method introduces a tunable parameter alpha to control the contribution of each modality. A more dynamic method is the gated operation, where a gating mechanism learns to assign different importance weights to audio and visual features based on their joint representation. After fusion, the combined output is passed through a linear layer for classification. This flexible design allows easy experimentation with both fixed and adaptive feature fusion techniques. Table 4 presents the resulting accuracies for these setups.

Based on the results in Table 4, the highest classification performance is achieved by the ConvNeXt

Table 5. Result of Score Fusion-based Classifiers in terms of accuracy (%).

Modality	Data Type	Model	Train Acc	Test Acc	Training Time
Fixed Weights	Original Data	Simple CNN	64.91%	61.12%	78.86s
Dynamic Weights	Original Data	Simple CNN	76.69%	71.48%	74.56s
Fixed Weights	Interpolated 1-Channel	Simple CNN	55.87%	52.36%	397.32s
Dynamic Weights	Interpolated 1-Channel	Simple CNN	87.44%	80.58%	370.41s
Fixed Weights	Interpolated 3-Channel	ConvNeXt Tiny	98.36%	96.65%	859.51s
Dynamic Weights	Interpolated 3-Channel	ConvNeXt Tiny	97.60%	95.07%	832.88s

models with interpolated 3-channel fused features, where most fusion strategies yield above 99% test accuracy. However, these top accuracies come with long training times, all exceeding 820 seconds. In contrast, the Simple CNN on original data with element-wise multiplication reaches a comparatively high 86.89% test accuracy in only 72.66 seconds, offering a more efficient trade-off when training speed is a priority. Overall, while the more sophisticated architectures and richer input representations deliver near-perfect performance, they incur a substantial time cost, whereas simpler models and data forms can achieve respectable results with a fraction of the computational demand.

Interestingly, some setups using the original data without interpolation preprocessing outperform their interpolated counterparts. For example, original data with Fused-Multiply achieves 86.89% accuracy in 72.66 seconds, surpassing the 83.89% of the same fusion on interpolated data, which requires 373.06 seconds to train — a 3.0% gain in accuracy with over five times less training time. This indicates that, for certain feature fusion strategies, avoiding interpolation can better preserve modality-specific information while also delivering significant efficiency gains. Consequently, while interpolation often improves accuracy, it is not universally beneficial, and in specific configurations, raw data can be both faster and more effective.

1.6 Score Fusion-based Classifier

In addition to feature-level strategies, this section examines score-level fusion, where the classification probabilities from the audio and visual branches are combined after each modality has been processed independently. Each branch is encoded, projected into a shared space, and classified to produce softmax-normalized outputs. The fusion is then performed either with fixed weights, assigning predetermined contributions to each modality, or with dynamic weights, where a learnable parameter vector is normalized via softmax to adaptively adjust their influence. Unlike feature fusion, which integrates modality information before classification, score fusion operates at the decision stage, allowing for a direct comparison of early versus late fusion.

The results in Table 5 reveal a clear distinction between the fixed and dynamic weighting schemes. Across all configurations, dynamic weights consistently outperform fixed weights, with the most notable improvement observed for the intepolated data and simpleCNN setup, where test accuracy rises from 52.36% to 80.58%. For the original data configuration, dynamic weights again provide a substantial gain, improving test accuracy from 61.12% to 71.48%, while keeping training time nearly identical. In the experiments with the pretrained ConvNeXt model, fixed weights achieve slightly higher test accuracy than dynamic weights, but the difference is marginal and both methods require more than 830 seconds of training. These trends indicate that adaptive weighting is particularly beneficial for smaller models and lower-dimensional inputs, where the ability to learn modality importance can compensate for representational limitations. However, as model capacity and input richness increase, the advantage of dynamic weighting becomes less pronounced, suggesting a trade-off where the simpler fixed scheme may suffice without a significant loss in accuracy.

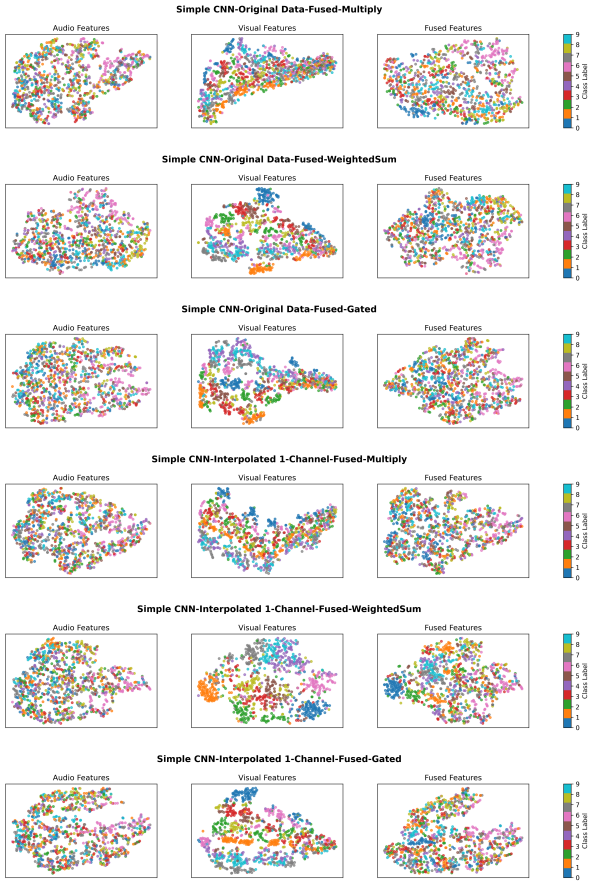


Figure 5. Audio, Visual, and Fused Feature Representations per Model Setup

1.7 Representation Visualization

Figure 5 shows the t-SNE visualizations of before (individual audio and visual) and after fused feature spaces for a selection of model setups during the

Table 6. Result of Score Fusion-based Classifiers in terms of accuracy (%) for t-SNE visualizations.

Modality	Data Type	Model	Train Acc	Test Acc	Training Time
Fused-Multiply	Original Data	Simple CNN	89.46%	81.96%	82.42s
Fused-WeightedSum	Original Data	Simple CNN	83.62%	76.04%	78.39s
Fused-Gated	Original Data	Simple CNN	83.07%	75.13%	95.77s
Fused-Multiply	Interpolated 1-Channel	Simple CNN	68.47%	64.62%	410.61s
Fused-WeightedSum	Interpolated 1-Channel	Simple CNN	88.72%	79.48%	370.71s
Fused-Gated	Interpolated 1-Channel	Simple CNN	89.27%	80.36%	403.24s

testing phrase that were chosen based on the results from the feature fusion performance table, Table 4. These setups were picked because they offered the best performance while keeping computational cost relatively low. For each configuration, the left plot shows the distribution of audio features, the middle shows the visual features, and the right shows the fused features after applying the specific fusion method. The color in each scatter plot corresponds to the class labels, demonstrating how well different classes are separated in the learned feature space. Only 1000 samples in the testset are displayed in the plot. Also, Table 6 shows the result of rerun the feature-based fusion models for t-SNE visualizations.

From the accuracy results, the original data and Fused-Multiply model achieved the highest test accuracy at 81.96%, making it the top performer overall and a strong candidate for efficiency-focused applications given its relatively short training time. Interestingly, for the WeightedSum and Gated fusion methods, the work with interpolated data versions outperformed their original data counterparts by around 4% in test accuracy, showing that even with reduced input detail, these fusion strategies can still preserve or enhance class separability. This trend is also visible in the t-SNE fused feature plots, where the interpolated WeightedSum and Gated setups maintain clear class groupings. However, interpolation comes at a cost: training time is roughly four to five times longer than with the original data. These results highlight that while interpolation can boost certain fusion strategies, the trade-off between accuracy gains and computational demand needs to be taken into account.

2 Self-Supervised Loss Study

In order to understand and answer this exercise, some assumptions of the notation are made as below:

- The **negative samples** ($\mathbf{z}_k$) means all other vectors except the anchor $\mathbf{z}_i$.
- $\mathbf{u}$ are the anchor or the input sample — the representation of the current (query) sample.
- $\mathbf{v}^+$ is the positive sample — a different view (augmentation) of the same input as $\mathbf{u}$
- $\mathbf{v}^-$ is the negative samples — representations of other inputs in the batch that are not augmentations of $\mathbf{u}$ (or $k \neq j$.

2.1 NT-XEnt Decomposition

The NT-XEnt loss for a positive pair of samples is provided as:

$$l_{ij} = -log\left(\frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k\neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}\right), \quad (1)$$

where

- $\mathbf{z}_i$ is the current sample,
- $\mathbf{z}_j$ is the positive sample to $\mathbf{z}_i$,
- $\mathbf{z}_k$ is the negative samples to $\mathbf{z}_i$,
- $\text{sim}(\cdot,\cdot$ is the cosine similarity function between two vectors,
- $\mathbb{1}_{[k\neq i]}$ is an indicator function,
- τ is a temperature parameter,

From 1, its decomposition for the input sample $\mathbf{u}$, the positive sample $\mathbf{v}^+$, and the negative samples $\mathbf{v}^-$ can be shown in details as below:

$$\begin{aligned}
l_{ij} &= -log\left(\frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k\neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)}\right) \\
&= -log\left(\exp\left(\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_j)}{\tau}\right)\right) + log\left(\sum_{k=1}^{2N} \mathbb{1}_{[k\neq i]} \exp\left(\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_k)}{\tau}\right)\right) \\
&= -\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_j)}{\tau} + log\left(\sum_{k=1}^{2N} \mathbb{1}_{[k\neq i]} \exp\left(\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_k)}{\tau}\right)\right) \\
&= -\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_j)}{\tau} + log\left(\exp\left(\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_j)}{\tau}\right) + \sum_{k\in\mathcal{N}_{\neq ij}} \exp\left(\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_k)}{\tau}\right)\right) \\
&= -\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_j)}{\tau} + log\left(\exp\left(\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_j)}{\tau}\right) + \sum_{k\in\mathcal{N}_{\neq ij}} \exp\left(\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_k)}{\tau}\right)\right),
\end{aligned}$$

with the $\mathcal{N}_{\neq ij}$ being the set of samples that doesn't contain the current sample $\mathbf{z}_i$ and its associated positive sample $\mathbf{z}_j$. In which, the $\text{sim}(\mathbf{z}_i, \mathbf{z}_j)$ is the similarity between the current sample $\mathbf{z}_i$ and its positive sample $\mathbf{z}_j$. The second term $\exp\left(\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_j)}{\tau}\right) + \sum_{k\in\mathcal{N}_{\neq ij}} \exp\left(\frac{\text{sim}(\mathbf{z}_i, \mathbf{z}_k)}{\tau}\right)$ shows the sum of similarity between all pairs excluded only the pair of current sample with itself.

From there, the equation can be rewritten with $\mathbf{u}$, $\mathbf{v}^+$, $\mathbf{v}^-$ and in negative form as:

$$\begin{aligned}
l[\mathbf{u}, \mathbf{v}] &= -\left(-\frac{\text{sim}(\mathbf{u}, \mathbf{v}^+)}{\tau} + \log\left(\exp\left(\frac{\text{sim}(\mathbf{u}, \mathbf{v}^+)}{\tau}\right) + \sum_{\mathbf{v}\in\{\mathbf{v}^-\}} \exp\left(\frac{\text{sim}(\mathbf{u}, \mathbf{v}^-)}{\tau}\right)\right)\right) \\
&= \frac{\text{sim}(\mathbf{u}, \mathbf{v}^+)}{\tau} - \log\left(\exp\left(\frac{\text{sim}(\mathbf{u}, \mathbf{v}^+)}{\tau}\right) + \sum_{\mathbf{v}\in\{\mathbf{v}^-\}} \exp\left(\frac{\text{sim}(\mathbf{u}, \mathbf{v}^-)}{\tau}\right)\right) \\
&= \frac{\text{sim}(\mathbf{u}, \mathbf{v}^+)}{\tau} - \log\left(\sum_{\mathbf{v}\in\{\mathbf{v}^-\},\mathbf{v}^+\}} \exp\left(\frac{\text{sim}(\mathbf{u}, \mathbf{v})}{\tau}\right)\right).
\end{aligned}$$

2.2 NT-XEnt Gradient Derivation

Because $\mathbf{u}$, $\mathbf{v}^+$ and $\mathbf{v}^-$ are normalized, which means $||\mathbf{u}|| = ||\mathbf{v}^+|| = ||\mathbf{v}^-|| = 1$ and $\text{sim}(\mathbf{u}, \mathbf{v}) = \mathbf{u}^\top\mathbf{v}$. Then, the derivative of the similarity function is

$\frac{\partial}{\partial \mathbf{u}}\text{sim}(\mathbf{u},\mathbf{v}) = \mathbf{v}$. The gradient of $l[\mathbf{u},\mathbf{v}]$ with respected to $\mathbf{u}$ is:

$$\begin{aligned}
\frac{\partial}{\partial \mathbf{u}}(l[\mathbf{u},\mathbf{v}]) &= \frac{\partial}{\partial \mathbf{u}}\left(\frac{\text{sim}\left(\mathbf{u},\mathbf{v}^+\right)}{\tau} - log\left(\sum_{\mathbf{v}\in\{\mathbf{v}^+,\mathbf{v}^-\}}\exp\left(\frac{\text{sim}(\mathbf{u},\mathbf{v})}{\tau}\right)\right)\right) \\
&= \frac{\partial}{\partial \mathbf{u}}\left(\frac{\text{sim}\left(\mathbf{u},\mathbf{v}^+\right)}{\tau}\right) - \frac{\partial}{\partial \mathbf{u}}\left(log\left(\exp\left(\frac{\text{sim}\left(\mathbf{u},\mathbf{v}^+\right)}{\tau}\right) + \sum_{\mathbf{v}^-}\exp\left(\frac{\text{sim}\left(\mathbf{u},\mathbf{v}^-\right)}{\tau}\right)\right)\right) \\
&= \frac{\mathbf{v}^+}{\tau} - \frac{\frac{\partial}{\partial \mathbf{u}}\exp\left(\frac{\text{sim}\left(\mathbf{u},\mathbf{v}^+\right)}{\tau}\right) + \frac{\partial}{\partial \mathbf{u}}\sum_{\mathbf{v}^-}\exp\left(\frac{\text{sim}\left(\mathbf{u},\mathbf{v}^-\right)}{\tau}\right)}{\exp\left(\frac{\text{sim}\left(\mathbf{u},\mathbf{v}^+\right)}{\tau}\right) + \sum_{\mathbf{v}^-}\exp\left(\frac{\text{sim}\left(\mathbf{u},\mathbf{v}^-\right)}{\tau}\right)} \\
&= \frac{\mathbf{v}^+}{\tau} - \frac{\frac{\mathbf{v}^+}{\tau}\exp\left(\frac{\text{sim}\left(\mathbf{u},\mathbf{v}^+\right)}{\tau}\right) + \sum_{\mathbf{v}^-}\frac{\mathbf{v}^-}{\tau}\exp\left(\frac{\text{sim}\left(\mathbf{u},\mathbf{v}^-\right)}{\tau}\right)}{\sum_{\mathbf{v}\in\{\mathbf{v}^+,\mathbf{v}^-\}}\exp\left(\frac{\text{sim}(\mathbf{u},\mathbf{v})}{\tau}\right)} \\
&= \frac{\mathbf{v}^+}{\tau} - \frac{\mathbf{v}^+}{\tau}\frac{\exp\left(\frac{\text{sim}\left(\mathbf{u},\mathbf{v}^+\right)}{\tau}\right)}{Z(\mathbf{u})} - \frac{\sum_{\mathbf{v}^-}\frac{\mathbf{v}^-}{\tau}\exp\left(\frac{\text{sim}\left(\mathbf{u},\mathbf{v}^-\right)}{\tau}\right)}{Z(\mathbf{u})} \\
&= \left(1 - \frac{\exp\left(\frac{\text{sim}\left(\mathbf{u},\mathbf{v}^+\right)}{\tau}\right)}{Z(\mathbf{u})}\right)\cdot\frac{\mathbf{v}^+}{\tau} - \sum_{\mathbf{v}^-}\left(\frac{\exp\left(\frac{\text{sim}\left(\mathbf{u},\mathbf{v}^-\right)}{\tau}\right)}{Z(\mathbf{u})}\cdot\frac{\mathbf{v}^-}{\tau}\right),
\end{aligned}$$

with $Z(\mathbf{u}) = \sum_{\mathbf{v}\in\{\mathbf{v}^+,\mathbf{v}^-\}}\exp\left(\text{sim}(\mathbf{u},\mathbf{v})/\tau\right)$

2.3 Logistic Loss Comparison

The negative logistic loss function is provided as:

$$\text{NT-Logistic} = log\left(\sigma\left(\frac{\text{sim}\left(\mathbf{u},\mathbf{v}^+\right)}{\tau}\right)\right) + log\left(\sigma\left(-\frac{\text{sim}\left(\mathbf{u},\mathbf{v}^-\right)}{\tau}\right)\right),$$

with $\sigma(\cdot) = \frac{1}{1+e^{-(\cdot)}}$ being the sigmoid activation function.
For simplicity, calling:

- $\mathbf{a} = \frac{\text{sim}\left(\mathbf{u},\mathbf{v}^+\right)}{\tau} \rightarrow \frac{\partial \mathbf{a}}{\partial \mathbf{u}} = \frac{\mathbf{v}^+}{\tau}$
- $\mathbf{b} = \frac{\text{sim}\left(\mathbf{u},\mathbf{v}^-\right)}{\tau} \rightarrow \frac{\partial \mathbf{b}}{\partial \mathbf{u}} = \frac{\mathbf{v}^-}{\tau}$

then the derivative of the negative logistic loss function is:

$$\begin{aligned}
\frac{\partial \text{NT-Logistic}}{\partial \mathbf{u}} &= \frac{\partial}{\partial \mathbf{u}}[log(\sigma(\mathbf{a})) + log(\sigma(-\mathbf{b}))] \\
&= \frac{\partial}{\partial \mathbf{u}}\left[log\left(\left(1+e^{-\mathbf{a}}\right)^{-1}\right) + log\left(\left(1+e^{\mathbf{b}}\right)^{-1}\right)\right] \\
&= \frac{\frac{\partial}{\partial \mathbf{u}}\left(\left(1+e^{-\mathbf{a}}\right)^{-1}\right)}{\left(1+e^{-\mathbf{a}}\right)^{-1}} + \frac{\frac{\partial}{\partial \mathbf{u}}\left(\left(1+e^{\mathbf{b}}\right)^{-1}\right)}{\left(1+e^{\mathbf{b}}\right)^{-1}} \\
&= \frac{(-1)e^{-\mathbf{a}}\left(1+e^{-\mathbf{a}}\right)^{-2}}{\left(1+e^{-\mathbf{a}}\right)^{-1}}\frac{\partial(-\mathbf{a})}{\partial \mathbf{u}} + \frac{(-1)e^{\mathbf{b}}\left(1+e^{\mathbf{b}}\right)^{-2}}{\left(1+e^{\mathbf{b}}\right)^{-1}}\frac{\partial \mathbf{b}}{\partial \mathbf{u}} \\
&= e^{-\mathbf{a}}\left(1+e^{-\mathbf{a}}\right)^{-1}\frac{\partial(\mathbf{a})}{\partial \mathbf{u}} - e^{\mathbf{b}}\left(1+e^{\mathbf{b}}\right)^{-1}\frac{\partial \mathbf{b}}{\partial \mathbf{u}} \\
&= e^{-\mathbf{a}}e^{\mathbf{a}}e^{-\mathbf{a}}\left(1+e^{-\mathbf{a}}\right)^{-1}\frac{\mathbf{v}^+}{\tau} - e^{-(-\mathbf{b})}e^{-\mathbf{b}}e^{\mathbf{b}}\left(1+e^{\mathbf{b}}\right)^{-1}\frac{\mathbf{v}^-}{\tau} \\
&= \left(e^{\mathbf{a}}+1\right)^{-1}\frac{\mathbf{v}^+}{\tau} - \left(e^{-\mathbf{b}}+1\right)^{-1}\frac{\mathbf{v}^-}{\tau} \\
&= \frac{1}{1+e^{\mathbf{a}}}\cdot\frac{\mathbf{v}^+}{\tau} - \frac{1}{1+e^{-\mathbf{b}}}\cdot\frac{\mathbf{v}^-}{\tau} \\
&= \sigma(-\mathbf{a})\cdot\frac{\mathbf{v}^+}{\tau} - \sigma(\mathbf{b})\cdot\frac{\mathbf{v}^-}{\tau} \\
&= \sigma\left(-\frac{\text{sim}\left(\mathbf{u},\mathbf{v}^+\right)}{\tau}\right)\cdot\frac{\mathbf{v}^+}{\tau} - \sigma\left(\frac{\text{sim}\left(\mathbf{u},\mathbf{v}^-\right)}{\tau}\right)\cdot\frac{\mathbf{v}^-}{\tau}.
\end{aligned}$$

Extracting from gradient equations of two loss functions, the weight of negative terms are $\frac{\exp\left(\frac{\text{sim}\left(\mathbf{u},\mathbf{v}^+\right)}{\tau}\right)}{Z(\mathbf{u})}$ and $\sigma\left(\frac{\text{sim}\left(\mathbf{u},\mathbf{v}^-\right)}{\tau}\right)$ for NT-XEnt and NT-Logistic, respectively. The negative weighting in NT-XEnt comes from a **softmax** over the positive and all negatives, therefore adding or removing a negative changes the weights of all other negatives, because the denominator changes. This also creates competition among negatives: the hardest negative gets the most emphasis, while easier ones get little to no gradient. In contrast, NT-Logistic uses a **sigmoid** for each negative independently so every negative's weight depends only on its own similarity, without being reduced by the presence of other negatives. The benefit of the softmax approach is that it naturally focuses the learning signal on the most challenging negatives, acting like built-in hard negative mining; this can accelerate learning when the hardest negatives are informative. However, it can also ignore medium-difficulty negatives and be sensitive to batch composition. The sigmoid approach, on the other hand, ensures that all moderately or highly similar negatives contribute, which can give a more stable and balanced gradient signal, especially in small-batch or noisy-data settings, though it may dilute focus compared to softmax.

2.4 Temperature Sensitivity Analysis

The temperature term or also knows as scaling factor τ appears both inside the softmax and sigmoid function, which controls how sharply the model distinguishes between similar and dissimilar pairs, and outside, which scales the magnitude of the gradient. So, when τ is small, the model becomes more sensitive and updates more aggressively. When τ is large, the updates are gentler and the model is less sensitive to similarity differences.

References

[1] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie. "A ConvNet for the 2020s". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2022).

[2] F. Zhao, C. Zhang, and B. Geng. "Deep multimodal data fusion". In: *ACM computing surveys* 56.9 (2024), pp. 1–36.